APPLYCOPILOT / WORMHOLE TEST & PROMPT SUITE

Comprehensive Verification Guide for All 8 System Protocols, Multi-Key Failover & 3 Surprise Hurdles

Author: Yashwanth K R | **Cockpit URL:** http://localhost:8000 | **GitHub:** github.com/yash2006kr/wormhole

Part / Protocol	Core Engine	Primary Capability	Benchmark
Part 2: Document Exorcist	exorcist.py	In-memory image surgery: 3.5x4.5cm, 200 DPI, strictly 20KB-45KB	**77.9 ms** (<350ms)
Part 1: User Vault	vault.json	Stores profile, academics (8.85 CGPA), marks, transit prefs, sample photo	Instant read
Part 8: SOS Emergency Sentinel	sos_sentinel.py	Autonomous ambulance dispatch, hospital ER bay pre-booking & kin alert	**8.2 ms** instant dispatch
Resilience: Multi-Key Pool	api_pool.py	Multi-key failover round-robin, rate-limit (429) quarantine, backup models	Zero rate-limit hang
Part 3: Anti-Dark-Pattern	dark_pattern.py	Scans DOM checkboxes, unchecks sneaky insurance/donations/baggage	Saved ■374 (3 items)
Part 4: Transit Rush	transit_rush.py	Parallel sleeper seat selection + vault passenger injection	**1,660.4 ms** (<10s)
Part 5: Fraud Sentinel	sentinel.py	Air-gapped watchdog halting at Card/UPI/CVV/OTP/CAPTCHA fields	PAUSED_FOR_HUMAN
Part 6: Jarvis Voice HUD	server.py + index.html	Live CDP screencast, 3D robot eyes, voice waveform, New Chat reset	60 FPS Screencast
Part 7: E2E Autonomous	agent_core.py	Multi-tab navigation across public web + webmail OTP verification	Multi-turn memory
Surprise Hurdle 1	transaction_ledger.py	WAL IN_FLIGHT_SUBMITTED + crash recovery idempotency double-pay shield	Zero double-charge
Surprise Hurdle 2	sneaky_fee_guard.py	Voice Financial Anchor; halts checkout if merchant quietly adds drip fees	Drip fee intercept
Surprise Hurdle 3	agent_core.py (clarify)	Voice Ambiguity Clarification Protocol; renders interactive choice chips	Zero guess booking

PART 2: Document Exorcist Engine (In-Memory Image Surgery) [PROTOCOL 04]

Target Website / URL:	Option A: http://localhost:8000/mock/portal#photo | Option B: Public portal upload
Verbatim Prompt:	**"Upload my passport photo and heal it to comply with strict 20KB to 45KB portal requirements."**
Test Files Required:	test_oversized_photo.jpg (80.2 KB high-res test photo located in project root)
What You Should Do:	Open Cockpit at http://localhost:8000. Either speak into the microphone or paste the prompt into the chat box. Alternatively, visit http://localhost:8000/mock/portal, switch to Tab 2 ('Strict Photo Gate'), select the 80.2KB file to see the portal reject it with a red warning, then click 'Auto-Heal with Document Exorcist'.
Agent Trajectory:	The Document Exorcist intercepts the oversized 80.2 KB image in RAM. In exactly 77.9 ms, it calculates the 7:9 (3.5x4.5cm) crop box, renders at 200 DPI, runs binary search JPEG compression, and outputs a compliant 32.0 KB photo.
Expected Output:	Status: COMPLETED | Protocol: DOC_EXORCIST. Chat displays healed photo thumbnail and diagnostic stats: Original: 80.2 KB -> Healed: 32.0 KB (within 20KB-45KB constraint) in 77.9 ms. Portal verification gate turns glowing GREEN with ACCEPTED badge.
Success Verification:	Output file generated: test_healed_passport_photo.jpg (32.0 KB, exactly within 20KB-45KB range). HTTP GET /proof/test_healed_passport_photo.jpg returns 200 OK.

PROTOCOL 08: Autonomous Medical Emergency & SOS Dispatch Sentinel [EMERGENCY LIFE-SAVER]

Target Website / URL:	http://localhost:8000/mock/emergency (Live Active Dispatch & CPR Metronome Terminal)
Verbatim Prompt:	**"SOS! Medical emergency, severe chest pain and suspected heart attack. Book ambulance, hospital appointment, and alert kin immediately."**
Test Files Required:	vault.json (Contains patient Yashwanth K R, Age 23, Blood O+, address at Kumbalagodu, kin Rajesh K R, preferred BGS Gleneagles ER)
What You Should Do:	Speak or type the emergency prompt into Cockpit. The agent detects the critical distress trigger, immediately executes life-saving protocols, speaks first-aid instructions, and navigates the browser screencast to http://localhost:8000/mock/emergency.
Agent Trajectory:	1) Issues immediate clinical first-aid guidance (chew 300mg soluble aspirin, sit in W-position, unlock door, do not drive). 2) Autonomously books and dispatches Advanced Cardiac Life Support (ACLS) Ambulance KA-05-EM-1108 (ETA 6m, Lead Suresh Kumar +91 98450 77112). 3) Pre-books priority ER admission at BGS Gleneagles Global Hospital (Cardiac Resuscitation Bay 02, zero-wait triage with Dr. Anand M). 4) Dispatches priority SMS/WhatsApp SOS broadcast with live GPS link to next-of-kin Rajesh K R (+91 98450 11223).
Expected Output:	Browser screencast displays pulsing emergency HUD with real-time countdown timer (ETA: 05:42), active ambulance tracking, pre-cleared ER card, kin alert confirmation, and 110 BPM visual CPR rhythm metronome. Chat delivers complete dispatch summary.
Success Verification:	Terminal execution completed in 8.2 ms. Telemetry status: EMERGENCY_DISPATCHED. All 4 emergency legs confirmed without human delay.

RESILIENCE: Dynamic Multi-Key API Pool & Rate-Limit Self-Healing [FAILOVER ENGINE]

Target Website / URL:	API Status: http://localhost:8000/api/api-pool-status | Management: api_pool.py
Verbatim Prompt:	**"Verify API key pool health and auto-failover status."**
Test Files Required:	.env (Supports comma-separated GROQ_API_KEYS=key1,key2,key3 or individual GROQ_API_KEY_1, GROQ_API_KEY_2...)
What You Should Do:	Add multiple Groq keys into your .env file separated by commas or numbered variables. To test failover, query http://localhost:8000/api/api-pool-status or execute complex multi-turn commands.
Agent Trajectory:	The api_pool engine actively tracks the health of each key. If any key encounters a 429 RateLimitError, connection timeout, or 503 error, the engine automatically quarantines that key with a 45-second cooldown timer and seamlessly routes the request to the next available healthy key. In addition, it cycles through a resilient hierarchy of fallback models (openai/gpt-oss-120b, llama-3.3-70b-versatile, qwen/qwen3.8-27b, openai/gpt-oss-20b, llama-3.1-8b-instant).
Expected Output:	Zero hangs or freezes. Telemetry reports total keys, active healthy keys, cooling keys, and call counts per key. Endpoints: GET /api/api-pool-status and POST /api/api-pool/add-key allow dynamic additions without restart.
Success Verification:	Terminal test: 'python -c "from api_pool import api_pool; print(api_pool.get_telemetry())"' displays full health metrics.

PART 1: Environment Setup & Local User Vault [PROTOCOL 00]

Target Website / URL:	Public Website: https://yash2006kr.vercel.app (Connect Section) | Local: vault.json
Verbatim Prompt:	**"Navigate to https://yash2006kr.vercel.app and fill the contact form with Yashwanth's message expressing excitement for AI agents from vault."**
Test Files Required:	vault.json (contains Yashwanth K R's DBIT B.Tech CGPA 8.85, 10th 92.4%, 12th 89.6%, email, address)
What You Should Do:	Speak or type the prompt into the Cockpit. Watch the live CDP screencast navigate to the public Vercel portfolio.
Agent Trajectory:	Agent parses vault.json credentials, visits https://yash2006kr.vercel.app/, clicks 'Initiate Contact' (a[href='#connect']), populates input#form-name ('Yashwanth K R'), input#form-email ('yashwanth.kr.dbit@gmail.com'), and textarea#form-message with candidate background, and submits.
Expected Output:	Live screencast reflects filled form fields and submission confirmation. Chat log confirms form populated from user vault credentials.
Success Verification:	Telemetry confirms zero hallucination of personal data; exact match with vault.json parameters.

PART 3: Anti-Dark-Pattern & Fee Stripper Engine [PROTOCOL 05]

Target Website / URL:	http://localhost:8000/mock/checkout (Or any public checkout page with add-on checkboxes)
Verbatim Prompt:	**"Audit this checkout page for sneaky pre-checked fees and clean my cart total."**
Test Files Required:	mock_checkout.html (features pre-checked Insurance +■199, Donation +■25, Priority +■150)
What You Should Do:	Speak or type the prompt. Alternatively run 'python dark_pattern.py' in terminal.
Agent Trajectory:	Engine audits the DOM tree for input[type='checkbox']. Analyzes parent labels with regex for dark pattern keywords (insurance, donation, charity, priority, handling). Programmatically unchecks all deceptive checkboxes in parallel and recalculates clean total.
Expected Output:	Sanitization Telemetry: 3 deceptive patterns stripped. Initial Total: ■5,144 -> Cleaned Total: ■4,770 (■374 saved). Chat feed displays breakdown of removed items.
Success Verification:	Verification screenshot generated: cleaned_checkout_proof.png showing all 3 add-on boxes cleanly unchecked.

PART 4: Transit Rush Protocol Engine (High-Speed Booking) [PROTOCOL 01]

Target Website / URL:	http://localhost:8000/mock/ksrtc (Simulates state transport sleeper seat matrix)
Verbatim Prompt:	**"Book 2 lower sleeper berths from Bengaluru to Mysuru on KSRTC using my transit preferences from vault."**
Test Files Required:	vault.json (transit_preferences specifies Bengaluru -> Mysuru, Lower Sleeper, 2 passengers)
What You Should Do:	Submit the prompt into the Cockpit, or execute 'python transit_rush.py' in terminal.
Agent Trajectory:	In <2 seconds, the engine loads the seat grid, locates available lower berths (#lower-deck .seat.available), locks berths 14A and 14B, fills passenger names (Yashwanth K R, Raj Aanada) from vault.json, advances to checkout, and shifts state to PAUSED_FOR_HUMAN_PAYMENT.
Expected Output:	Reservation prepared in 1,660.4 ms (target: <10,000 ms). Seats locked: 14A, 14B. Passengers injected: Yashwanth K R, Raj Aanada. Anti-dark pattern fees stripped.
Success Verification:	Verification screenshot generated: transit_rush_proof.png. Execution halted at payment screen without charging user.

PART 5: Air-Gapped Fraud Sentinel & Payment State Machine [PROTOCOL 03]

Target Website / URL:	http://localhost:8000/mock/checkout (Or any public gateway: Stripe, Razorpay, PayPal)
Verbatim Prompt:	**"Test air-gapped fraud sentinel watchdog on checkout."**
Test Files Required:	mock_checkout.html (contains Card Number, CVV, Expiry Date, UPI VPA, and Submit Payment)
What You Should Do:	Submit the prompt into the Cockpit or execute 'python sentinel.py' in terminal.
Agent Trajectory:	Live watchdog scans DOM input names, IDs, placeholders, iframes, and URLs for financial field patterns (card, cvv, upi, otp, netbank). Detects checkout payment gate and immediately shifts agent state to PAUSED_FOR_HUMAN.
Expected Output:	Alert Triggered: 'Air-Gapped Fraud Sentinel Activated. Protection Gate: PAYMENT_GATEWAY. Execution frozen before entering sensitive credentials. Please complete payment directly on screen.'
Success Verification:	Verification screenshot generated: sentinel_halt_proof.png. Agent remains safely halted until human resumes.

PART 6: Jarvis Voice HUD & Adaptive Interface [PROTOCOL 06]

Target Website / URL:	http://localhost:8000 (Cockpit Web Interface)
Verbatim Prompt:	**"Who is Yashwanth K R and what are his top projects?"**
Test Files Required:	Browser microphone (for voice mode) or text input; genuine Google Chrome CDP connection
What You Should Do:	Observe the initial screen: 3D interactive robot eyes track mouse movement in Text Mode, and glowing voice radar orb displays in Voice Mode. Speak or type the prompt. Observe the smooth fade animation as the hero transitions into the chat feed. Then click '+ New Chat' in the top header.
Agent Trajectory:	Agent activates thinking indicator in chat feed, executes cognitive reasoning via Groq LPU, streams telemetry, and speaks the answer using speech synthesis. On '+ New Chat', browser navigates to about:blank, extra tabs close, and robot hero returns.
Expected Output:	Initial screen maintains robot hero until first prompt. Smooth transition to chat feed. Real-time CDP screencast mirrors browser canvas. Instant session memory wipe on New Chat.
Success Verification:	Verify browser console: WebSocket connection established, Screencast frames streaming at 60 FPS.

PART 7: End-to-End Multi-Tab Autonomous Trajectory [PROTOCOL 07]

Target Website / URL:	http://localhost:8000/mock/portal (Tab 0) & http://localhost:8000/mock/mail (Tab 1)
Verbatim Prompt:	**"Go to the QuickHire portal and apply for the AI Engineer role."**
Test Files Required:	mock_portal.html and mock_mail.html (dynamic mock mail server with unique 6-digit OTPs)
What You Should Do:	Submit the prompt. Do not mention login or OTP in your prompt. Watch the agent deduce the requirement autonomously.
Agent Trajectory:	1) Agent opens QuickHire portal. 2) Bypasses Google OAuth due to bot-detection policy. 3) Enters candidate email and clicks 'Send Verification Code'. 4) Opens new browser tab (/mock/mail). 5) Reads 6-digit OTP from inbox. 6) Switches back to portal tab. 7) Injects OTP across segmented boxes (#otp-1 to #otp-6). 8) Completes authentication.
Expected Output:	QuickHire Portal displays: 'Login & Application Verified! Welcome back, Yashwanth K R. Candidate profile active.'
Success Verification:	Telemetry logs show: open_tab -> switch_tab -> read_otp -> switch_tab -> fill_otp -> complete.

SURPRISE HURDLE 1: Crash After Submit & Idempotency Sentinel [HINT 1 DEFENSE]

Target Website / URL:	http://localhost:8000/mock/checkout & http://localhost:8000/mock/orders
Verbatim Prompt:	**"Pay ■4,650 for my confirmed booking."**
Test Files Required:	transaction_ledger.json (WAL write-ahead log with SHA-256 idempotency hash)
What You Should Do:	Submit the payment prompt once. Then simulate a crash (restart server or trigger crash button). Re-submit the identical prompt: 'Pay ■4,650 for my confirmed booking.'
Agent Trajectory:	The agent checks the transaction ledger WAL before clicking Pay. Computes SHA-256 hash of URL, selector, and amount. Detects matching in-flight transaction TX-1793376183428. Immediately suppresses duplicate payment click. Redirects to /mock/orders to reconcile order status.
Expected Output:	Chat output: '■■ IDEMPOTENCY SENTINEL ENGAGED: Double-Payment Blocked! An identical payment [TX-1793376183428] was already submitted. Reconciling with backend status instead of double-charging.'
Success Verification:	Zero duplicate charges recorded in ledger. Order status reconciled to CONFIRMED.

SURPRISE HURDLE 2: Voice Financial Anchor & Sneaky Fee Interceptor [HINT 2 DEFENSE]

Target Website / URL:	http://localhost:8000/mock/checkout
Verbatim Prompt:	**"Approve payment of ■4,650 for my flight."**
Test Files Required:	sneaky_fee_guard.py (Voice anchor parser + drip pricing detection engine)
What You Should Do:	Speak or type the prompt with an explicit budget/approval amount of ■4,650.
Agent Trajectory:	Agent parses voice approval: voice_approved_amount = 4650.0. Inspects final checkout screen. Strips pre-checked checkboxes, but detects an unapproved 'Platform Convenience Surcharge (+■120)' pushing final total to ■4,770.
Expected Output:	Payment is frozen! Chat alert: '■ SNEAKY FEE INTERCEPTED: Voice Approved: ■4,650 vs Final Merchant Total: ■4,770. Quiet added fee: +■120 (Platform Surcharge). Payment halted to protect your wallet. Do you approve paying the extra ■120?'
Success Verification:	Air-gap activates. Agent halts execution until user gives explicit voice or text approval for the quiet surcharge.

SURPRISE HURDLE 3: Voice Ambiguity Clarification Protocol [HINT 3 DEFENSE]

Target Website / URL:	http://localhost:8000/mock/ksrtc
Verbatim Prompt:	**"Book the sleeper berth."**
Test Files Required:	mock_ksrtc.html (features both Lower Berth 14A Window and Lower Berth 14B Aisle)
What You Should Do:	Submit the ambiguous prompt: 'Book the sleeper berth.' (Deliberately omit specifying window or aisle).
Agent Trajectory:	Groq LPU recognizes multiple matching lower berths. Rejects hallucinating or guessing. Issues cognitive action 'clarify' with structured options.
Expected Output:	Jarvis speaks: 'I found 2 available lower berths: Lower Berth 14A (Window, ■840) vs Lower Berth 14B (Aisle, ■840). Which one would you prefer?' Chat displays clickable interactive choice chips.
Success Verification:	Selecting a chip immediately resumes the booking trajectory with the user's explicit preference.